

Assignment 7

This assignment encourages you to explore on your own and learn how to use the scikit-learn API for common ML tasks. Some of the material was covered in brief towards the end of the regression lecture.

Read the first 6 sections of [Chapter 5](#) of the [Data science handbook](#). These are well written and provide a nice overview of many important concepts. There are also several nice code samples that are worth studying. You will also find the [scikit-learn user guide](#) very handy to skim or refer.

1. Consider the pendulum data provided in the `data/pendulum.txt` file and used in our regression material. Use `sklearn.model_selection.train_test_split` to split the data into 75% training data and 25% test data. Look at the [sklearn user guide](#) to read more about the `train_test_split` functionality. Now use the training data to determine the best fit coefficients using a linear regression (this was done at the end in the regression notebook I shared). Recall that we had a linear model for l vs. t^2 . Then use the trained model to fit the test data. Calculate the error on the test data using the `model.score` method where `model` is your `LinearRegression` model. Read the documentation of `model.score` to understand what it is computing.
2. Convert the pendulum data into a `pandas.DataFrame` object with suitable columns for length `l`, and the square of the time period, `tsq`. Read the [data as table](#) section to see how to use the data frame as your input features matrix and target array. Use this to once again fit a linear model between the length and square of the time period. Note that this is a rather trivial exercise but this should show you how to deal with data frames and use them in scikit-learn.
3. Now just to see how you can compute a cross-validation score, use `sklearn.model_selection.cross_val_score` on the first problem. Use $k = 4$ for this, i.e. use the keyword argument `cv=4`. To understand this better, see the [cross-validation user documentation from scikit-learn](#). Note that this should not take you more than 3-4 lines of code after you solve the previous problem.
4. Read the [section on feature engineering](#) from the Python data science handbook. Pay particular attention to the example where the `PolynomialFeatures` preprocessor was highlighted. Use this

to redo the linear model for the pendulum data where you use the original feature as t and not the square of the values. Recall, that originally we had taken t^2 as the dependent/response variable (Y), and the length as the independent/feature variable x . Now, reverse this and treat the length as the target variable and the time period as the input feature. Then use a `PolynomialFeatures` preprocessor to calculate the new polynomial features with polynomial order 2. Then use this new data to fit the model and look at the new coefficients. Evaluate the error in the model as in the first problem.

5. Use the `sklearn.pipeline.make_pipeline` function documented [here](#) to implement the previous problem. Use the `PolynomialFeature` and `LinearRegression` classess to make the pipeline. Note that this is again just a few lines of code.
6. Consider the data in the file `student_performance.csv` with categorical features. First study the dataset (use a spreadsheet to view the data for example) and clean up the column names with spaces and replace them with underscores using pandas (and not the spreadsheet application!). For example `"math score"` should be `"math_score"`. You may also rename some long column names suitably. Our plan is to see if the math scores can be well predicted with a linear regression model using all the available data (excluding the math scores of course). Clearly this is multiple linear regression with categorical variables given the nature of the data. To do this, first use the `pd.get_dummies` function to encode all categoricals as binary indicator variables and leaving one of the values (use the keyword argument `drop_first=True`). Then fit a linear model with `LinearRegression`. Remember to drop the `"math_score"` from the dataset to do this properly (hint: use `data_frame.drop()` to drop columns and return a new data frame). Using all the data except the `math_score` column as input variables, construct a linear model. Initially use all the data and perform the fit. Plot the predictions versus the actual value using the same input values, plot the reading score on the x-axis and plot the predicted and actual math scores on the y-axis. Perform a test/train split and plot the predicted vs. actual values again for this new fit.
7. Repeat the above but this time use `sklearn.preprocessing.OneHotEncoder` to preprocess the data and fit a linear model. Compare the results.
8. Read the section on [hyperparameters and model validation](#). Consider the following function given there to make some data.

```
import numpy as np

def make_data(N, err=1.0, rseed=1):
    # randomly sample the data
    rng = np.random.RandomState(rseed)
    X = rng.rand(N, 1) ** 2
    y = 10 - 1. / (X.ravel() + 0.1)
    if err > 0:
        y += err * rng.randn(N)
    return X, y
```

Use this function to make data for 40 points as done in the book. Then repeat the calculations done there to see what happens when you increase the order of the polynomial and plot the validation curve. Note that the import in the book `from sklearn.learning_curve import validation_curve` is out-of-date and should now be `from sklearn.model_selection import validation_curve`.

9. Use the approach from the previous problem to look at our original pendulum data and see what happens with increasing degree of polynomials for this data. In particular plot the validation curve for this data as done in the previous problem.
10. Look at the `sklearn.datasets.make_blobs` function from the scikit-learn documentation. Use this to make blobs with two and three centers each (see the `centers` keyword argument). Plot the centers for two and three blobs (in separate figures). Use about 50 points for each blob, so 100 for 2 centers and 150 for 3. You can see examples of this in the [Naive Bayes section](#) of the data science handbook.
11. Use `make_blobs` to create two clusters of points (say 200 points). Use `sklearn.naive_bayes.GaussianNB` to classify the points. Split the data into 80% for training and 20% for testing. After training compute the following for the **training and test data** separately (note that once trained, you can use `model.predict_proba` to get the probabilities for each class). For example, you should compute the test accuracy and training accuracy and report both.
 - a. Show a confusion matrix. Hint: `sklearn.metrics.confusion_matrix`
 - b. Compute the accuracy, precision, recall, and f1 scores. Hint: see `sklearn.metrics`

- c. Compute the ROC/AUC using the `auc`, `roc_curve`, `RocCurveDisplay` from `sklearn.metrics`
12. Experiment with different values of the cluster standard deviation in the `make_blobs` function and see how this affects the scores and results for the previous problem.
13. Use `sklearn.linear_model.LogisticRegression` to repeat the problem 8 above and see how it performs.